淺度機器學習分類器比較 -- 使用資料 face_data，並將其設定為ds1

學號：411079060

姓名：石欽鈺

完成日期：2023/5/12

作品目標： 在這個數據龐大的時代，多變量、大數據、即時反應已是很常見的場景。此時，機器學習是解決這些需求的工具，而我們想知道不同的分類器在不同資料面前的性能，於是進行 `評比實驗` 來探討機器學習中常見的分類方法，包括 `多元羅吉斯回歸（MLR）`、`支援向量機（SVM）`、以及神經網路（`Neural Network`） ，同時我們也想知道用原始資料與主成分資料於分類時的差異，無論是準確度的變化還是整體時間耗費多寡。因此，我們還會將資料分成原始資料與主成分資料，除此之外，部分資料還有做看看降噪處理，並去觀察他的性能：

1. 介紹並認識三種常見的統計分類方法：多元羅吉斯回歸、支援向量機、神經網路。
2. 評比實驗：
   - 使用三種不同的資料集，包括大小、變數多寡、資料型態等不同特徵的資料，來測試三種分類器的準確度。
   - 準確度的優化: 包括但不限於參數的調整（如正規化、gamma、degree等）、計算方式的選擇（如演算法、激活函數等）、以及資料預處理（如標準化、PCA / SVD、降噪等）。
3. 針對不同資料 `原始 & 0.85 比例的主成份` ，找出最佳準確度組合，並去比較後哪種分類器在影像分類任務中表現最佳， `不同資料型態在分類時會產生什麼樣的差異` ，並給出相應的建議和結論。 </font>

In [ ]:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_csv('face_data.csv')
X = np.array(df.iloc[:, :-1]) # Exclude the last column (label)
y = np.array(df.iloc[:, -1]) # Label column

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30)

scaler = StandardScaler()
X_train_ = scaler.fit_transform(X_train)
X_test_ = scaler.fit_transform(X_test)
```

**交叉驗證版本的Logistic: LogisticRegressionCV **

- 自動選擇 `最優的正則化強度參數 C` 要嘗試的 C 值的範圍（會在這個範圍內搜索最佳的 C 值）
  - 這裡用 np.logspace(-5, 5, 20)，在 `指數` 尺度上均勻分布 20 個 C 值。

ds1 -- lr -- std

In [ ]:

```
from sklearn.linear_model import LogisticRegressionCV
from sklearn.metrics import accuracy_score
import numpy as np

opts = dict(tol=1e-6, max_iter=int(1e6), verbose=0)
solver = 'lbfgs'

Cs = np.logspace(-5, 5, 20)

clf_originalCV = LogisticRegressionCV(solver=solver, Cs=Cs, **opts)
clf_originalCV.fit(X_train_, y_train)  # input data must be (n_samples, n_features)
y_pred = clf_originalCV.predict(X_test_)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}\n")
```

```
Accuracy: 96.67%
```

ds1 -- lr -- PCA

In [ ]:

```
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# PCA transformation
pca = PCA(n_components=0.85).fit(X_train_)
Z1_train = pca.transform(X_train_)
Z1_test = pca.transform(X_test_)
```

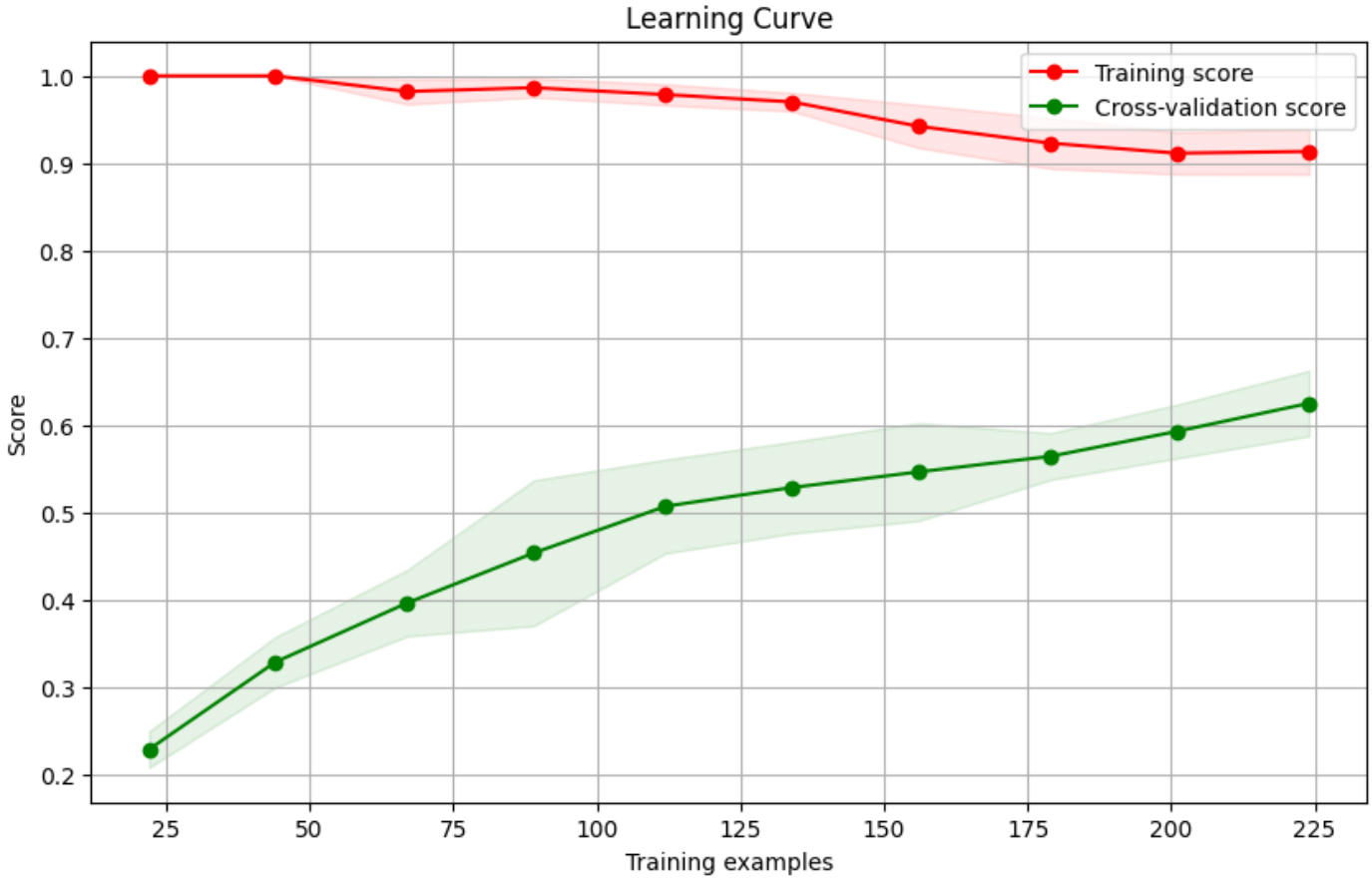
```
opts = dict(tol=1e-6, max_iter=int(1e6), verbose=0)
solver = 'lbfgs'  # 'lbfgs' is the default

clf_PCA = LogisticRegression(solver=solver, **opts)
clf_PCA.fit(Z1_train, y_train)
y_pred = clf_PCA.predict(Z1_test)

# Report accuracy on test data
print(f"Accuracy using clf_PCA.score: {clf_PCA.score(Z1_test, y_test):.2%}\n")
```

```
Accuracy using clf_PCA.score: 95.83%
```

- lbfgs是最適合的演算法，大於newton-cg的95.00%，並且直接使用LogisticRegressionCV來尋找最好正規化參數，最後獲得95.83%的準確度

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_PCA, Z1_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



1. **定義了一個降噪函數** `denoise_gaussian` ：
   - 輸入：灰度圖像
   - 對該圖像應用高斯濾波器進行降噪
   - 高斯濾波器的核大小為 (5, 5)，標準差為 0。
   - 輸出：降噪圖像
2. **對訓練集和測試集圖像進行降噪處理並展平為一維向量**：
   - 對降噪後的圖像進行 `展平` (flatten)，二維圖像數組 --> 一維向量
   - `Z_train` 和 `Z_test`：降噪後的特徵向量

ds1 -- lr -- denoise with std

In [ ]:

```
from sklearn.linear_model import LogisticRegression
import cv2

# 適用於灰度圖像的高斯濾波
def denoise_gaussian(image):
    # 應用高斯濾波器
    denoised_image = cv2.GaussianBlur(image, (5, 5), 0)
    return denoised_image

# 對圖像進行降噪處理並展平為一維向量
Z_train = [denoise_gaussian(image).flatten() for image in X_train_]
Z_test = [denoise_gaussian(image).flatten() for image in X_test_]

solver = 'lbfgs'  # 'lbfgs' is the default

C_value = 0.1
clf_denoise = LogisticRegression(solver=solver, C=C_value, tol=1e-6, max_iter=int(1e6), verbose=0)
clf_denoise.fit(Z_train, y_train)
y_pred = clf_denoise.predict(Z_test)
accuracy = clf_denoise.score(Z_test, y_test)  # 計算準確度
print(f"Accuracy using clf_denoise.score: {accuracy:.2%}\n")
```

```
Accuracy using clf_denoise.score: 96.67%
```

- liblinear 0.9417 < lbfgs 0.9667 = newton-cg
- 雖然 lbfgs = newton-cg 有著相同的準確度，但前者的時間需求很短，但後者的時間卻長達10分鐘，比較之下前者較好
- 降噪後的資料準確度雖然沒有如降噪前還要高，但差距非常小

ds1 -- svm

In [ ]:
```
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 1  # SVM regularization parameter
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.3, **opts)
# clf_svm = SVC(kernel="poly", degree=3, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

clf_svm.fit(X_train, y_train)
predictions = clf_svm.predict(X_test)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.60      0.75      0.67         4
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         5
           3       0.50      1.00      0.67         1
           4       1.00      0.75      0.86         4
           5       1.00      1.00      1.00         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       1.00      1.00      1.00         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       1.00      1.00      1.00         1
          16       1.00      1.00      1.00         1
          17       1.00      1.00      1.00         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       1.00      1.00      1.00         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
          26       1.00      1.00      1.00         3
          27       1.00      1.00      1.00         5
          28       1.00      1.00      1.00         2
          29       1.00      1.00      1.00         2
          30       1.00      1.00      1.00         1
          31       1.00      0.67      0.80         3
          32       1.00      1.00      1.00         1
          33       1.00      1.00      1.00         1
          34       1.00      1.00      1.00         2
          35       1.00      1.00      1.00         3
          36       1.00      1.00      1.00         4
          37       0.80      1.00      0.89         4
          38       1.00      1.00      1.00         3
          39       1.00      1.00      1.00         2

    accuracy                           0.97       120
   macro avg       0.97      0.97      0.97       120
weighted avg       0.98      0.97      0.97       120
```

- 用 rbf.poly 的話超級低，無論 gamma & degree 設多少都幾乎在0.02-0.0.5之間
- 不過使用linear的kernel都能有高達0.97的準確度，代表原始資料應該是線性可分，沒有過度複雜的

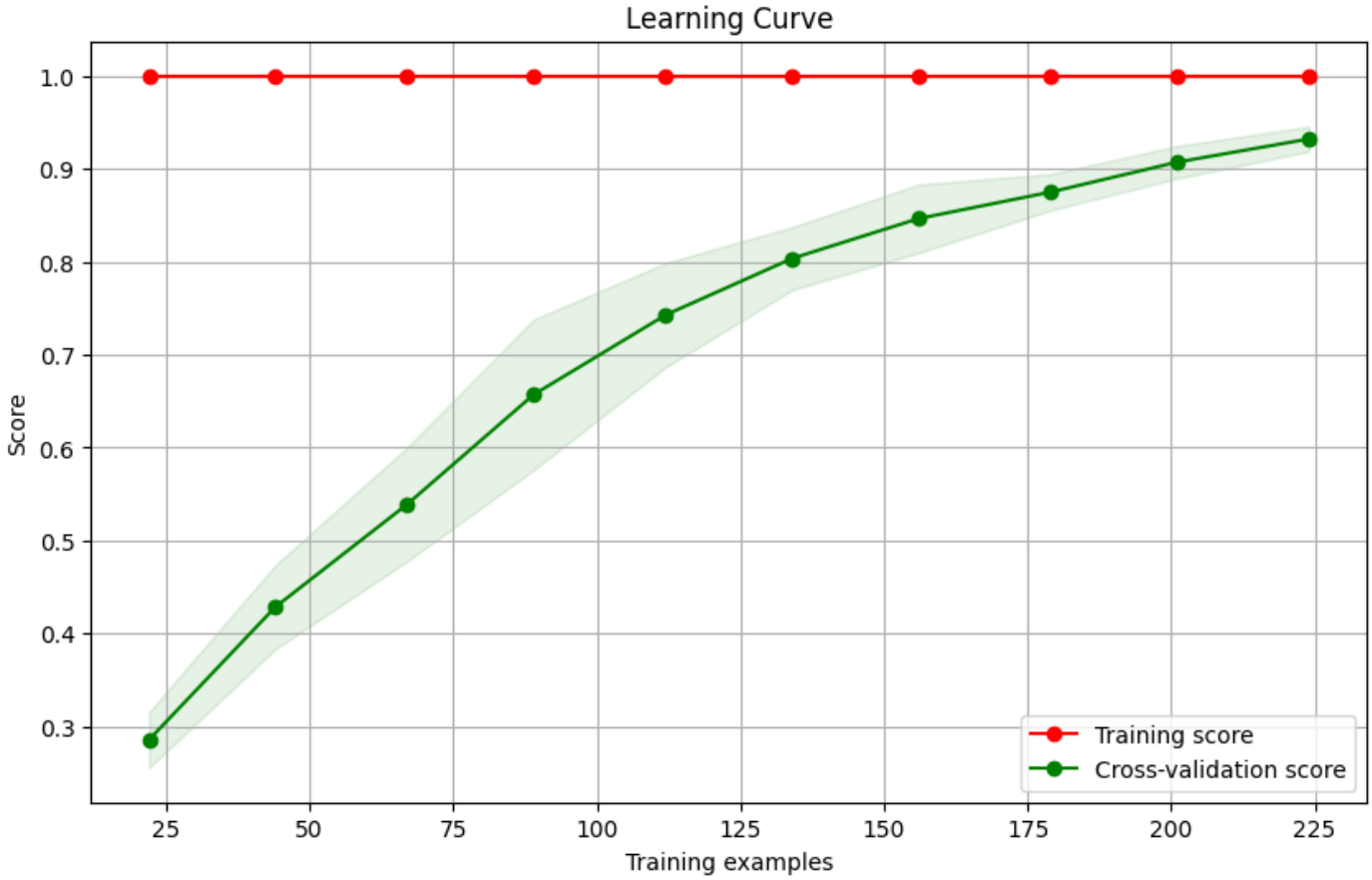
ds1 -- svm -- PCA

In [ ]:
```
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.2, **opts)
# clf_svm = SVC(kernel="poly", degree=1, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(Z1_train, y_train)

# Make predictions
predictions = clf_svm.predict(Z1_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.67      1.00      0.80         4
           1       1.00      1.00      1.00         4
           2       1.00      0.80      0.89         5
           3       1.00      1.00      1.00         1
           4       1.00      0.75      0.86         4
           5       1.00      1.00      1.00         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       1.00      1.00      1.00         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       1.00      1.00      1.00         1
          16       1.00      1.00      1.00         1
          17       1.00      1.00      1.00         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       0.75      1.00      0.86         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
          26       1.00      1.00      1.00         3
          27       1.00      1.00      1.00         5
          28       1.00      1.00      1.00         2
          29       1.00      1.00      1.00         2
          30       1.00      1.00      1.00         1
          31       1.00      1.00      1.00         3
          32       1.00      1.00      1.00         1
          33       1.00      1.00      1.00         1
          34       1.00      1.00      1.00         2
          35       1.00      1.00      1.00         3
          36       1.00      1.00      1.00         4
          37       1.00      1.00      1.00         4
          38       1.00      1.00      1.00         3
          39       0.50      0.50      0.50         2

    accuracy                           0.97       120
   macro avg       0.97      0.97      0.97       120
weighted avg       0.97      0.97      0.97       120
```

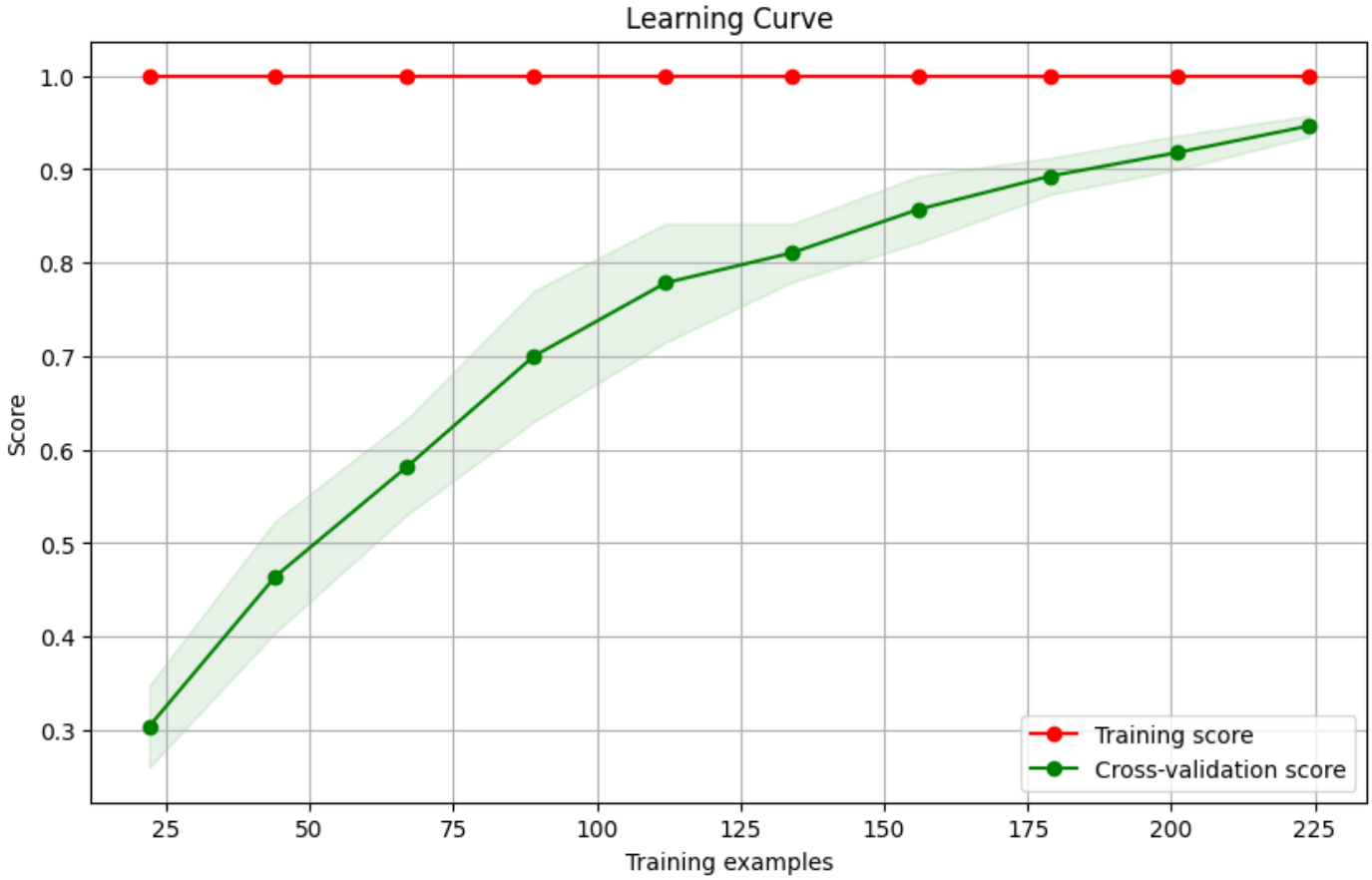
In [ ]:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_svm, X_train_, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_svm, Z1_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



ds1 -- svm -- denoise

In [ ]:

```
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.2, **opts)
# clf_svm = SVC(kernel="poly", degree=3, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(Z_train, y_train)

# Make predictions
predictions = clf_svm.predict(Z_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      0.75      0.86         4
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00         1
           4       1.00      0.75      0.86         4
           5       1.00      1.00      1.00         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       0.67      1.00      0.80         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       0.50      1.00      0.67         1
          16       1.00      1.00      1.00         1
          17       0.83      1.00      0.91         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       1.00      1.00      1.00         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
```

```
           26       1.00      1.00      1.00         3
           27       1.00      1.00      1.00         5
           28       1.00      1.00      1.00         2
           29       1.00      1.00      1.00         2
           30       1.00      1.00      1.00         1
           31       1.00      1.00      1.00         3
           32       1.00      1.00      1.00         1
           33       1.00      1.00      1.00         1
           34       1.00      1.00      1.00         2
           35       1.00      1.00      1.00         3
           36       1.00      1.00      1.00         4
           37       1.00      1.00      1.00         4
           38       1.00      1.00      1.00         3
           39       0.50      0.50      0.50         2

    accuracy                           0.97       120
   macro avg       0.96      0.97      0.96       120
weighted avg       0.97      0.97      0.97       120
```

by chatgpt:

- relu：線性整流函數，通常在深度學習中表現良好。
- tanh：雙曲正切函數，也是一種常見的激活函數。
- identity：恆等函數，對於不需要非線性轉換的任務可以使用。 ###
- sgd：隨機梯度下降，可以使用不同的變體，如 momentum、nesterov_momentum 等。
- adagrad：自適應學習率的梯度下降算法。
- rmsprop：根據過去梯度的均方根調整學習率的算法。
- lbfgs：L-BFGS 算法，基於拟牛頓法的優化器。

ds1 -- nn

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (120,)  # one hidden layer with 30 units
activation = 'logistic'
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000)
solver = 'adam'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(X_train, y_train)

# Make predictions
predictions = clf_MLP.predict(X_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.67      1.00      0.80         4
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00         1
           4       1.00      0.75      0.86         4
           5       0.80      1.00      0.89         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       1.00      1.00      1.00         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       1.00      1.00      1.00         1
          16       1.00      1.00      1.00         1
          17       1.00      1.00      1.00         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       1.00      1.00      1.00         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
          26       1.00      1.00      1.00         3
          27       1.00      1.00      1.00         5
          28       1.00      1.00      1.00         2
          29       1.00      1.00      1.00         2
          30       0.50      1.00      0.67         1
```

```
           31       1.00      0.67      0.80         3
           32       1.00      1.00      1.00         1
           33       1.00      1.00      1.00         1
           34       1.00      1.00      1.00         2
           35       1.00      1.00      1.00         3
           36       1.00      1.00      1.00         4
           37       1.00      1.00      1.00         4
           38       1.00      1.00      1.00         3
           39       1.00      0.50      0.67         2

    accuracy                           0.97       120
   macro avg       0.97      0.97      0.96       120
weighted avg       0.98      0.97      0.97       120
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
```

- logistic 比 tanh 好大約 0.1

ds1 -- nn -- PCA

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (120,)  # one hidden layer with 30 units
activation = 'logistic'
# alpha = 0.1  # regularization parameter
# beta_1 = 0.9  # exponential decay rate for estimates of first moment vector in adam
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000,
       )
solver = 'lbfgs'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(Z1_train, y_train)

# Make predictions
predictions = clf_MLP.predict(Z1_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         4
           1       1.00      1.00      1.00         4
           2       0.83      1.00      0.91         5
           3       1.00      1.00      1.00         1
           4       1.00      0.75      0.86         4
           5       1.00      1.00      1.00         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       1.00      1.00      1.00         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       0.50      1.00      0.67         1
          16       1.00      1.00      1.00         1
          17       1.00      1.00      1.00         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       1.00      1.00      1.00         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
          26       1.00      1.00      1.00         3
          27       1.00      1.00      1.00         5
          28       1.00      1.00      1.00         2
          29       1.00      1.00      1.00         2
          30       1.00      1.00      1.00         1
          31       1.00      1.00      1.00         3
          32       1.00      1.00      1.00         1
          33       1.00      1.00      1.00         1
          34       1.00      1.00      1.00         2
          35       0.75      1.00      0.86         3
          36       1.00      1.00      1.00         4
          37       1.00      1.00      1.00         4
          38       1.00      1.00      1.00         3
```

```
           39       1.00      0.50      0.67         2

    accuracy                           0.97       120
   macro avg       0.98      0.97      0.97       120
weighted avg       0.98      0.97      0.97       120
```

ds1 -- nn -- denoise

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (120,)  # one hidden layer with 30 units
activation = 'logistic'
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000)
solver = 'adam'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(Z_train, y_train)

# Make predictions
predictions = clf_MLP.predict(Z_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.80      1.00      0.89         4
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00         1
           4       1.00      0.75      0.86         4
           5       1.00      1.00      1.00         4
           7       1.00      0.75      0.86         4
           8       1.00      1.00      1.00         4
           9       1.00      1.00      1.00         4
          10       1.00      1.00      1.00         2
          11       1.00      1.00      1.00         4
          12       1.00      1.00      1.00         2
          13       1.00      1.00      1.00         4
          14       1.00      1.00      1.00         3
          15       1.00      1.00      1.00         1
          16       1.00      1.00      1.00         1
          17       1.00      1.00      1.00         5
          18       1.00      1.00      1.00         3
          19       1.00      1.00      1.00         5
          20       1.00      1.00      1.00         3
          21       1.00      1.00      1.00         4
          22       1.00      1.00      1.00         4
          23       1.00      1.00      1.00         3
          24       1.00      1.00      1.00         2
          25       1.00      1.00      1.00         4
          26       1.00      1.00      1.00         3
          27       1.00      1.00      1.00         5
          28       1.00      1.00      1.00         2
          29       1.00      1.00      1.00         2
          30       1.00      1.00      1.00         1
          31       1.00      1.00      1.00         3
          32       1.00      1.00      1.00         1
          33       1.00      1.00      1.00         1
          34       1.00      1.00      1.00         2
          35       0.75      1.00      0.86         3
          36       1.00      1.00      1.00         4
          37       1.00      1.00      1.00         4
          38       1.00      1.00      1.00         3
          39       1.00      1.00      1.00         2

    accuracy                           0.98       120
   macro avg       0.99      0.99      0.99       120
weighted avg       0.99      0.98      0.98       120
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
```

ds1 -- nn -- learning curve

In [ ]:

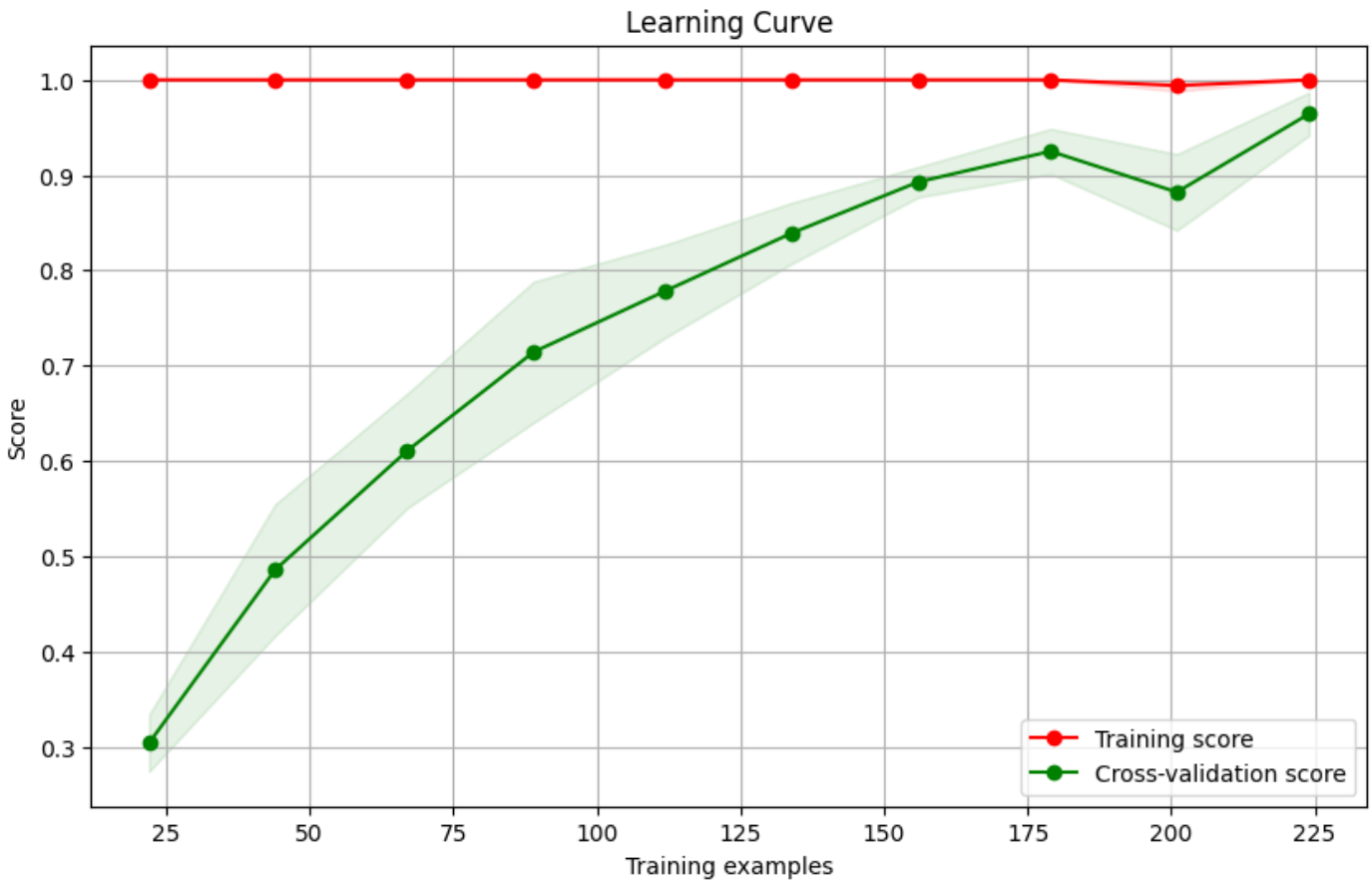
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve
```

```python
# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_MLP, X_train_, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
```

```
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
```

```
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multi
layer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the
optimization hasn't converged yet.
  warnings.warn(
```

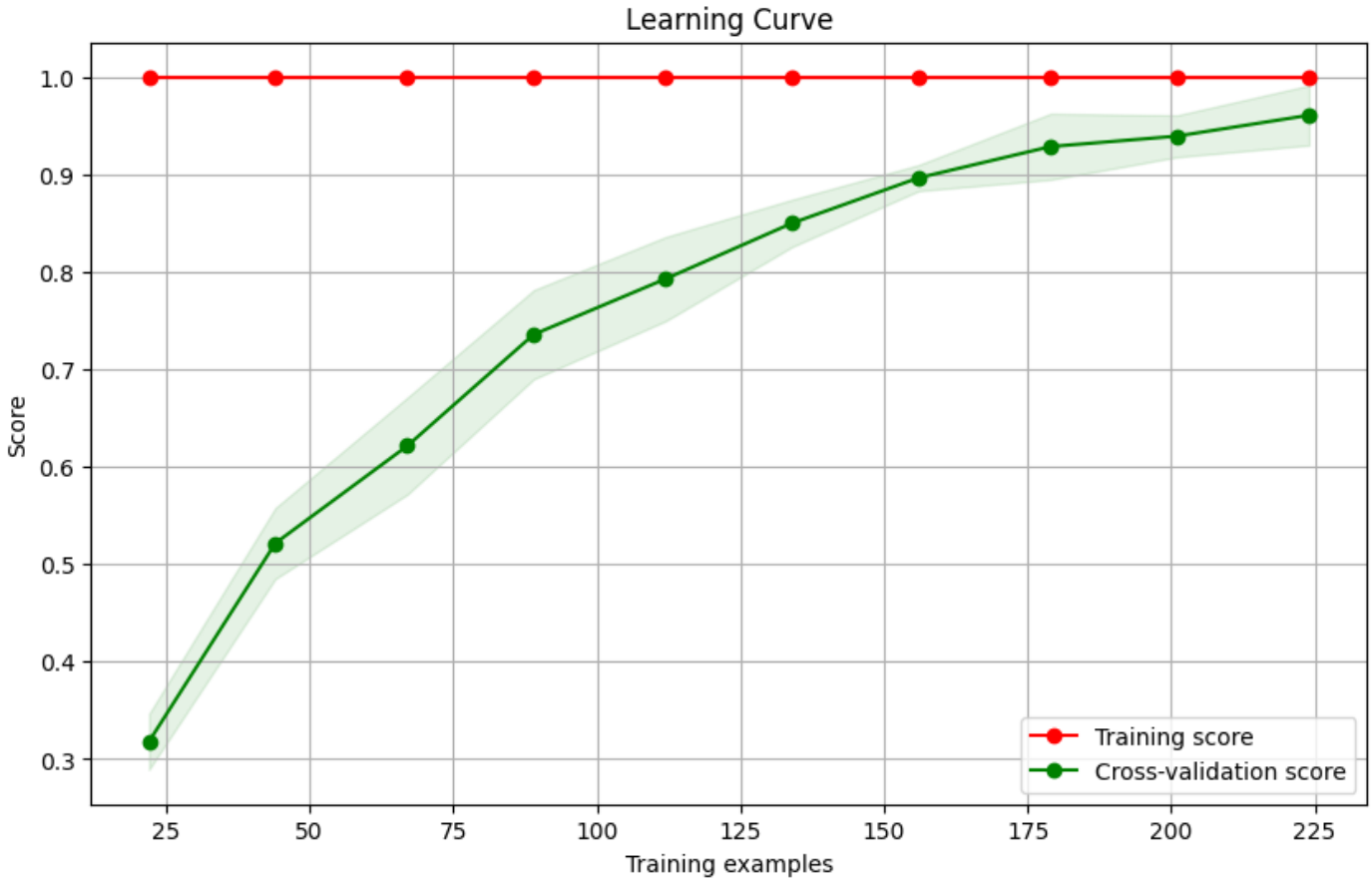


ds1 -- nn -- pca -- learning curve

In [ ]:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_MLP, Z1_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
```

```
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



結論：從以上評比實驗可知

- 羅吉斯迴歸（Logistic Regression）
  - 標準化資料：96.67%
  - PCA資料：95.83%
  - denoise資料：96.67%
- 支援向量機（Support Vector Machine, SVM）
  - 標準化資料：97%
  - PCA資料：97%
  - denoise資料：97%
- 神經網路（Multi-layer Perceptron, MLP）
  - 標準化資料：97%
  - PCA資料：97%
  - denoise資料：98%

比較

1. 運用羅吉斯分類器，我先用交叉驗證法來搜尋正規化參數，範圍訂在np.logspace(-5, 5, 20)，並採lbfgs演算法，產出96.67%的最佳準確度。在0.85比例的主成份資料中，我們也用相同設定（沒有正規化參數）來跑，一樣以lbfgs表現最好，大於newton-cg的95.00%，其執行速度也比較快一些。 而主成分資料的執行速度比原始資料還要少一半，但準確度卻沒有相差太多，蠻直得做主成分資料預處理的。 再來，從練習曲線圖片中我們可以發現，隨著樣本數越大，原始資料的訓練分數有些許下降，且訓練分數一直高於驗證分數，有可能有過度擬合的問題，不過驗證分數有隨著樣本數增加而增加，是理想的現象。在這邊我多做了降噪的預處理方式M而他花了很短的時間達到和原始資料一樣的準確度96.67%，感覺是蠻適合的預處理方式。

2. 在svm分類器中，我皆選用linear演算法，並將正規化參數設為1。在最佳組合下，標準化、pca和降噪資料皆獲得0.97的準確度，比羅吉斯分類器還高且穩定，而花的時間也較短。 透過學習曲線圖我們可以發現，隨著樣本數增加，無論是原始資料還是主成分資料最終驗證分數和訓練分數都可以達到蠻理想的狀態，且比神經元再更穩定一些。

3. 在神經網路中，我將原始資料與pca資料選用相同的激活函數logistic，相同的隱藏曾神經元數量120，不同的演算法，一個是adam，一個是lbfgs。其中以pca資料的macro avg較高(多1%)，他擁有97%的macro avg。 不過，降噪資料卻在這之中表現最好，除了有98%的準確度之外， macro avg梗達到99%。 透過學習曲線圖我們可以發現，隨著樣本數增加，在神經網路分類器下，無論是原始資料還是主成分資料最終驗證分數和訓練分數可以達到差不多的水準，是最理想的狀態，比在羅吉斯分類器下還要好。

建議 看起來神經網路和支援向量機都能夠給這筆資料很好的準確度，無論資料是主成分還是原始資料，在這些分類器的表現上基本上差異微乎其微，當然如果用主成份處理方式，可能使執行效率提高。 如果資料打算做降噪處理的話，神經元網路又更適合一些，且在學習曲線下我們可以發現，svm和神經元分類器真的蠻適合這筆資料的，不過也需備足足夠的樣本數，而svm可能再更穩定一些。